

IT5005 Artificial Intelligence
Tutorial 5
Tree and Adversarial Search

1. Sudoku is a popular number puzzle that works as follows: we are given a 9×9 square grid; some squares have numbers, while some are blank. Our objective is to fill in the blanks with numbers from 1–9 such that each row, column and the highlighted 3×3 squares contain no duplicate entries (see Figure 1). Solving Sudoku puzzles is often modeled as a CSP (which we will cover later in the course). Consider, however, the problem of *generating* Sudoku puzzles. Consider the following procedure: start with a completely full number grid (see Figure 2), and iteratively make some squares blank. We continue blanking out squares as long as the resulting puzzle can be completed in only one way.

Questions:

- a. Give the representation of a state in this problem;
- b. Using the state representation defined above, specify the initial state and goal state.
- c. Define its actions; and
- d. Using the state representation and actions defined above, specify the transition model/function T (i.e., when each of the actions defined above is applied to a current state, what is the resulting next state?).

2	5			3		9		1
	1				4			
4		7				2		8
		5	2					
				9	8	1		
	4				3			
			3	6			7	2
	7							3
9		3				6		4

Figure 1. A simple Sudoku puzzle

2	5	8	7	3	6	9	4	1
6	1	9	8	2	4	3	5	7
4	3	7	9	1	5	2	6	8
3	9	5	2	7	1	4	8	6
7	6	2	4	9	8	1	3	5
8	4	1	6	5	3	7	2	9
1	8	4	3	6	9	5	7	2
5	7	6	1	4	2	8	9	3
9	2	3	5	8	7	6	1	4

Figure 2. The solution to the puzzle in Figure 1.

2. Consider a class managed by an instructor. The instructor allocates one day to meet with students to discuss a final project; the day has t time slots in it denoted $S = \{s_1, s_2, \dots, s_t\}$. Every student i indicates their availability denoted by a subset $A_i \subseteq S$. Our objective is to schedule all students for a meeting, while respecting their availability constraints. Your goal is to model this as a search problem, with the state space defined as a partial (valid) allocation of some of the students into the time slots.

Suppose there are three time-slots, and three students whose availability is given below. Assume that whenever you need to choose a student to add to a schedule, you do so in the order (A)lice (B)ob and then (C)laire. In addition, when choosing a time slot, ties are always broken in order of $s_1 \succ s_2 \succ s_3$.

	s_1	s_2	s_3
Alice	✓	✓	✓
Bob	✓		
Claire		✓	

- Give a trace of the BFS algorithm in the style of Figure 3. That is, show the search tree at each stage (all repeated states are eliminated).
- Give a similar trace of the DFS search algorithm.
- Which of these two search algorithms is better for this problem? Why? Is one search strategy always better than the other in general?

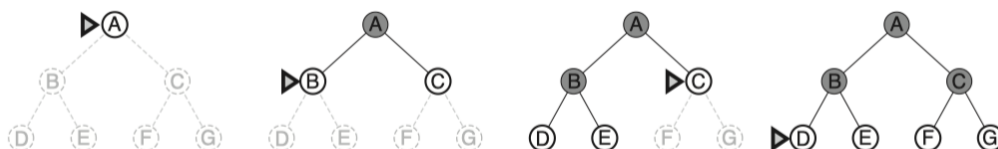


Figure 3. BFS on a simple binary tree. At each stage the node to be expanded is indicated by a marker.

3. Figure 4 below shows the familiar 8-puzzle where we slide tiles around until the numbers re in order when read from left to right, top to bottom:

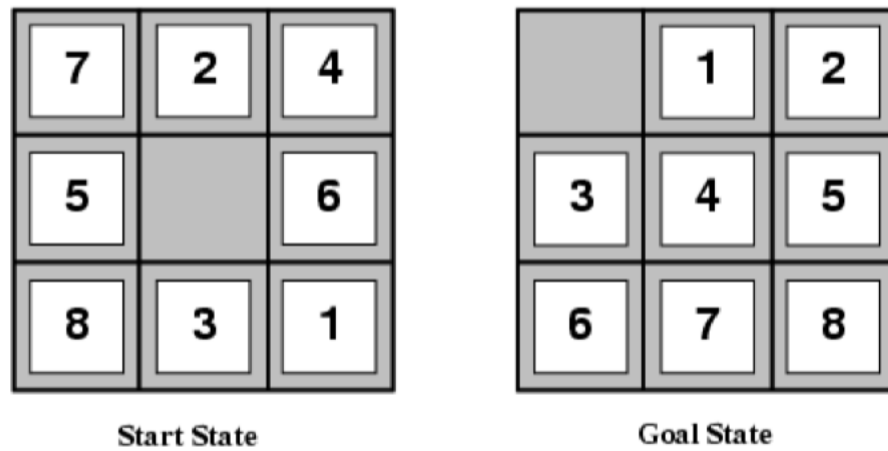


Figure 4. The 8-Puzzle

- a. We start with two heuristics:

$$h_1 = \# \text{ of misplaced tiles}$$

$$h_2 = \sum_i^n |x_{i,a} - x_{i,t}| + |y_{i,a} - y_{i,t}|$$

Where $(x_{i,a}, y_{i,a})$ is the actual position of a tile, and $(x_{i,t}, y_{i,t})$ is its target position. The top left position is (1, 1) and the bottom right position is (3, 3). This is traditionally called a "Manhattan Distance".

Are both heuristics admissible? Why or why not?

- b. Consider two more heuristic functions:

$$h_3 = \frac{h_1 + h_2}{2}$$

$$h_4 = h_1 + h_2$$

Are h_3 and h_4 admissible? Why or why not?

- c. We consider the idea of dominance. A heuristic h_2 is said to be dominant over h_1 if $\forall n, h_2(n) \geq h_1(n)$ if both h_2 and h_1 are admissible. Is h_2 dominant over h_1 ? Compare the dominance of h_3 and h_4 with respect to h_2 and h_1

4. Consider the minimax search tree shown in Figure 5 below:

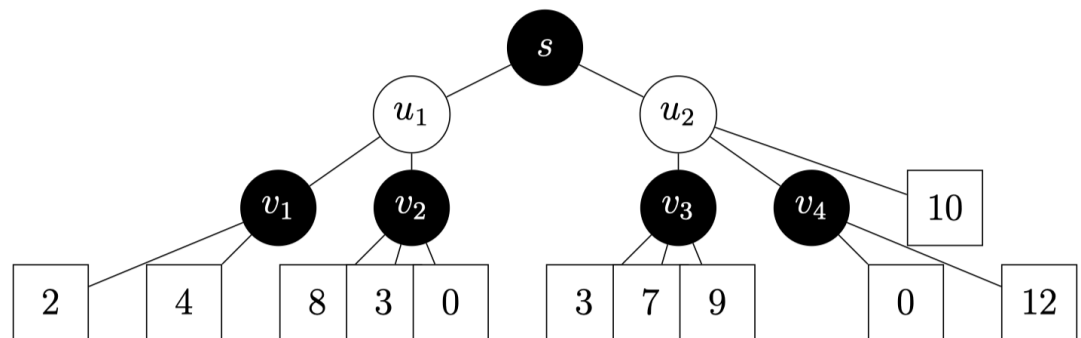


Figure 5. Minimax Search Tree

In the figure, black nodes are controlled by the MAX player, and white nodes are controlled by the MIN player. Payments (terminal nodes) are squares; the number within denotes the amount that the MIN player pays to the MAX player (an amount of 0 means that MIN pays nothing to MAX). Naturally, MAX wants to maximize the amount they receive, and MIN wants to minimize the amount they pay.

Suppose that we use the α - β pruning algorithm, reproduced in Figure 6 below.

```

function ALPHA-BETA-SEARCH(state) returns an action
   $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$ 
  return the action in ACTIONS(state) with value  $v$ 



---


function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for each  $a$  in ACTIONS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 
    if  $v \geq \beta$  then return  $v$ 
     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
  return  $v$ 



---


function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow +\infty$ 
  for each  $a$  in ACTIONS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 
    if  $v \leq \alpha$  then return  $v$ 
     $\beta \leftarrow \text{MIN}(\beta, v)$ 
  return  $v$ 

```

Figure 6. $\alpha - \beta$ pruning search

- Assume that we iterate over nodes from right to left; mark with an 'X' all ARCS that are pruned by α - β pruning, if any.
- Does your answer change if we iterate over nodes from left to right?